MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Information Technology

Guru Gobind Singh Indraprastha University, Delhi

MINOR PROJECT (ES-451)

Third Defense — Progress Report

ConstructTrack

Automated Construction Progress Monitoring, Delay Prediction and Risk Reporting System

Submitted by

S. No.	Name	Enrolment No.
1	Ashish Upadhyay	06714803123
2	Yash Rajput	06814803123
3	Sahil Rao	07614803123

Team No.: 54 Semester: 7th Session: 2026–27

Faculty Guide: Dr. Seema Kalonia

October 2026

Table of Contents

1. Project Overview.....1
1.1 Objectives.....1
2. Status of Work.....1
2.1 Work completed since the previous defense.....2
3. System Architecture.....3
3.1 Technology stack.....3
3.2 Data model.....4
4. Module Details.....4
4.1 Computer-vision pipeline.....4
4.2 Progress analytics.....4
4.3 Delay prediction and schedule-risk engine.....4
4.4 Automated reporting.....5
5. Screenshots.....5
6. Results and Validation.....6
6.1 Software verification.....6
6.2 Scenario evaluation of the delay and risk engine.....6
7. Deployment.....7
8. Research Paper Status.....7
9. Remaining Work and Timeline.....8
10. Future Scope.....8
References.....8

1. Project Overview

Title: ConstructTrack — Automated Construction Progress Monitoring, Delay Prediction and Risk Reporting System.

Domain: Web development, computer vision, and construction informatics (data analytics for project management).

Construction progress is still widely tracked in spreadsheets, site-visit notes and loose collections of photographs. Because planned-versus-actual figures and photographic evidence are reviewed separately and irregularly, early signs of schedule slippage are often noticed late, and every stakeholder report has to be assembled by hand. ConstructTrack is a web-based system that centralises a project's progress records and site photographs in one place and builds three analytical layers on top of them: (i) computer-vision analysis of site images using a YOLO11 object detector, kept strictly as supporting visual evidence; (ii) progress analytics that compare recorded progress with the planned schedule over real elapsed time; and (iii) a deterministic, explainable delay-prediction and schedule-risk engine. All of this is consolidated into an automated report that can be viewed on the web or downloaded as a PDF.

1.1 Objectives

1. Centralise each construction project's schedule, progress history and site images in one system.
2. Analyse uploaded site images with a real object-detection model and store the structured results.
3. Compute progress analytics (current completion, variance against plan, historical trend) from recorded data.
4. Predict a likely completion date and delay from the project's own progress velocity.
5. Produce a documented, deterministic 0–100 schedule-risk score with factor-level explanations.
6. Generate structured project reports and a downloadable PDF without manual data assembly.
7. Keep AI-derived visual evidence clearly separate from deterministic, server-computed project metrics.

2. Status of Work

All planned functional modules are implemented, integrated and running end to end. The work since the previous defense focused on hardening the system for a live hosted demonstration, validating the delay and risk engine on structured scenarios, and preparing the research paper. Table 1 summarises module status.

Table 1: Module-wise implementation status

Module	Key functionality	Status
Project management	Create, edit and delete projects with planned start/end dates, status and location	Complete
Progress tracking	Planned vs. actual records; automatic deviation and status (ahead / on track / at risk / delayed)	Complete
Image management	Upload JPEG/PNG/WEBP site photos; server-side format sniffing; private storage; gallery and timeline	Complete
AI analysis service	FastAPI + OpenCV + YOLO11n; detections, object counts, confidence, annotated image	Complete
Progress analytics	Current completion, variance, real elapsed-time trend, explicit insufficient-data states	Complete
Delay prediction & risk	Estimated completion date, predicted delay, 0–100 risk score with bands, delay-probability heuristic	Complete
Reporting	Structured report snapshot from a single project ID; print-friendly web view; server-generated PDF	Complete
Dashboard	Portfolio KPIs: total/active projects, average completion, at-risk count	Complete
Cloud deployment	Vercel (web) + Supabase (DB, storage) + Hugging Face Space (AI service)	Configured
Research paper	Draft on explainable schedule-risk scoring with CV as supporting evidence	Draft ready

2.1 Work completed since the previous defense

- **Deployment readiness:** added a Vercel build pipeline that generates the Prisma client and applies database migrations automatically; containerised the AI service (CPU-only PyTorch, model weights baked into the image) for free hosting on a Hugging Face Space.
- **Security:** optional shared-secret authentication (X-API-Key) between the web application and the publicly hosted AI service; secrets are server-side only.
- **Robustness:** client-side downscaling of large phone photographs so uploads stay within serverless request limits; extended timeouts for AI analysis and PDF generation; locally bundled fonts so builds do not depend on external font downloads.
- **Image linking:** site photos can now be linked to a specific progress record at upload time, as stated in the synopsis objectives.
- **Demo data:** rewrote the seed data so planned progress follows each project's own timeline, producing four realistic scenarios (behind schedule, on plan, ahead of plan, stalled).
- **Evaluation:** ran the delay and risk engine on structured scenarios and compared it with a deviation-only baseline (Section 6).
- **Research paper:** prepared a first full draft in IEEE conference format.

3. System Architecture

The system consists of two independently deployable services. The Next.js application owns the user interface, API routes, database access and storage integration. The Python FastAPI service performs only image preprocessing and YOLO11 inference; it receives a short-lived signed image URL and never holds database or storage credentials.

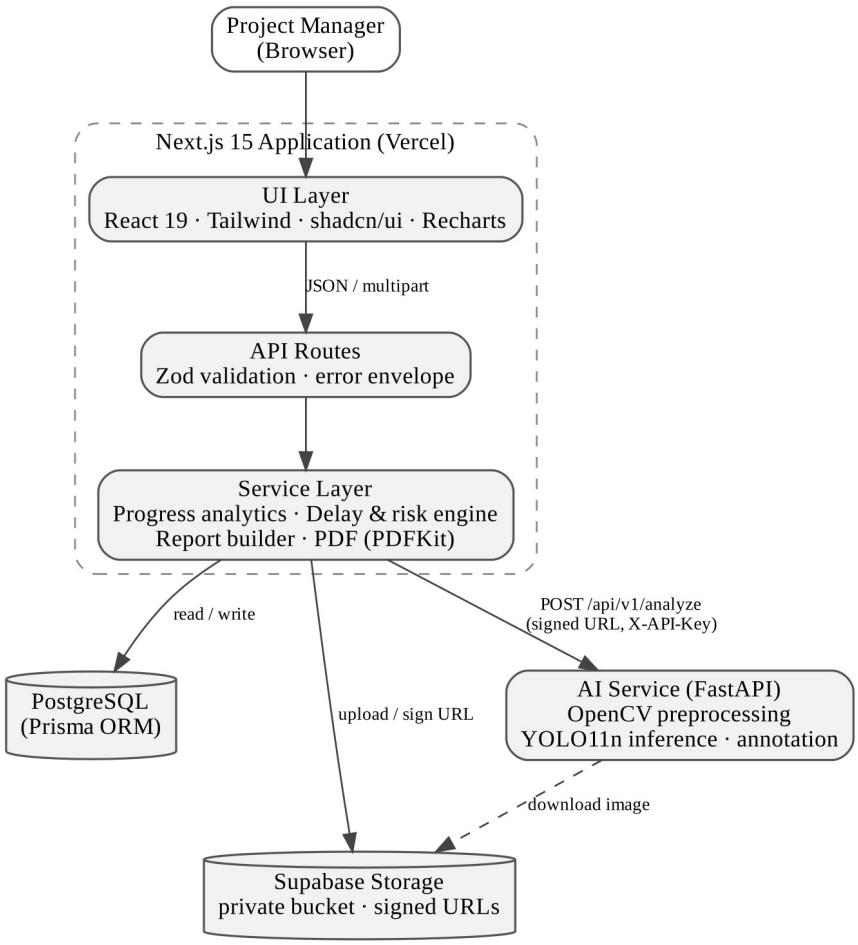


Figure 1: System architecture

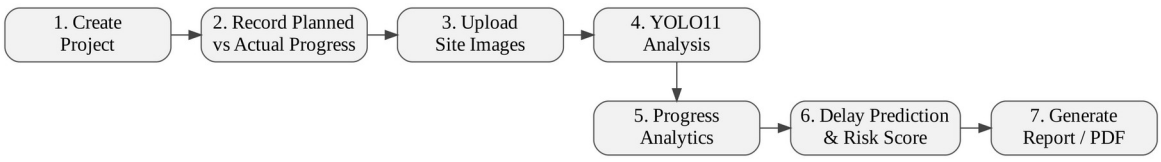


Figure 2: End-to-end workflow

3.1 Technology stack

Layer	Technologies
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS 4, shadcn/ui (Radix UI), Recharts
Backend	Next.js API routes, Zod validation, Prisma ORM 6
Database & storage	PostgreSQL; Supabase Storage (private bucket, short-lived signed URLs)
AI / computer vision	Python 3.11, FastAPI, Ultralytics YOLO11n (COCO-pretrained), OpenCV (headless), PyTorch, NumPy
Reporting	PDFKit (server-side PDF generation)

Layer	Technologies
Testing & tooling	Node test runner with tsx (61 tests), pytest (32 tests), ESLint, TypeScript strict type-checking
Deployment	Vercel, Supabase, Hugging Face Spaces (Docker)

3.2 Data model

The PostgreSQL schema (managed through Prisma migrations) contains seven entities: **User** (project manager), **Project**, **ProgressRecord** (one planned-vs-actual observation per date), **ConstructionImage** (metadata only; binaries live in storage), **AIAnalysis** (one-to-one with an image: model version, detections, object counts, average confidence, processing time, annotated-image path), **DelayPrediction** (per-record baseline estimate) and **Report** (a persisted JSON snapshot of a generated report). Foreign keys cascade on project deletion so no orphaned records remain.

4. Module Details

4.1 Computer-vision pipeline

When a user clicks Analyse on an image, the API marks the image as processing (rejecting a concurrent second run), generates a short-lived signed URL and calls the AI service. The service downloads the image with a size cap and magic-byte format check, decodes it with OpenCV, resizes it so the longest side is at most 1280 px, and runs YOLO11n inference (confidence threshold 0.25, IoU 0.45). Detections are aggregated into per-class counts and an average confidence, and an annotated copy with bounding boxes is returned. The web application persists the result and stores the annotated image separately, leaving the original untouched.

Scope boundary: YOLO11n is a general-purpose detector trained on the 80 COCO classes (people, vehicles, and similar). It is not a construction-progress model, so its output is never converted into a completion percentage and never feeds the delay or risk calculations. It is presented as supporting visual evidence, for example a change in detected workers or vehicles between two site visits.

4.2 Progress analytics

For each record, deviation = actual − planned. The latest record (by date) gives the current completion, variance and status (ahead ≥ +3 points; at risk ≤ −3; delayed ≤ −10). The historical trend uses real elapsed calendar time between the first and last records that carry an actual value, so irregular reporting intervals still give a correct rate. Fewer than two such records produces an explicit "trend unavailable" state instead of a fabricated value.

4.3 Delay prediction and schedule-risk engine

The engine answers three questions using only the project's stored dates and progress records:

```
averageDailyProgress  = (latestActual − earliestActual) / elapsedDays
requiredDailyProgress = (100 − latestActual) / daysUntilExpectedEnd
velocityRatio         = averageDailyProgress / requiredDailyProgress
```

```
estimatedCompletion   = latestDate + (100 − latestActual) /
averageDailyProgress
predictedDelayDays    = max(0, estimatedCompletion − expectedEndDate)
```

No completion date is produced when the rate is zero, negative or unknown. The schedule-risk score combines four factor scores (each 0–100) with fixed, documented weights:

Table 2: Schedule-risk factors and weights

Factor	Weight	Scoring rule
Schedule variance	30%	Deviation ≥ 0 → 10; [−5, 0) → 40; [−15, −5) → 65; < −15 → 90
Velocity adequacy	30%	Velocity ratio ≥ 1.0 → 10; [0.8, 1.0) → 35; [0.5, 0.8) → 65; < 0.5 → 90
Predicted schedule delta	25%	Predicted delay as a share of planned duration; 30% or more → 100
Historical trend	15%	Change in deviation across the last three records (improving / stable / worsening)

```
riskScore = round(0.30·V + 0.30·S + 0.25·D + 0.15·T)
```

Bands: 0–24 LOW, 25–49 MODERATE, 50–74 HIGH, 75–100 CRITICAL. A separate "Estimated Delay Probability" uses 0.40·V + 0.40·S + 0.20·T and is always labelled as a heuristic, not a statistically validated probability. Data sufficiency is reported as INSUFFICIENT (0–1 usable records, no prediction made), LIMITED (2), ADEQUATE (3–4) or STRONG (5+).

4.4 Automated reporting

A report is generated from a single project ID. The server collects the project, progress history, images and AI results, runs the same analytics and prediction functions used by the dashboard, and stores the composed result as a JSON snapshot. The report contains nine sections: executive summary, project overview, progress analytics, construction image summary, AI computer-vision analysis, delay prediction, risk analysis, methodology, and limitations and disclaimer. The same snapshot renders as a print-friendly web page and as a PDF generated on the server with PDFKit. Missing values are printed as "Data unavailable" rather than as placeholder numbers.

5. Screenshots

[Insert screenshots from the running application before printing.]

- Figure 3: Dashboard — portfolio KPIs
- Figure 4: Project detail — progress KPIs and planned-vs-actual chart
- Figure 5: Construction image analysis — YOLO11 detections and annotated image
- Figure 6: Delay prediction and schedule-risk section with factor explanations
- Figure 7: Generated report (web view) and downloaded PDF

6. Results and Validation

6.1 Software verification

The web application passes TypeScript strict type-checking, ESLint and a production build. The automated suites contain 61 unit tests for the web application (progress analytics, delay prediction, AI-evidence notes, report analytics, report composition and PDF generation) and 32 pytest tests for the AI service (preprocessing, aggregation, annotation, schema, health and analysis endpoints, including an end-to-end YOLO11 inference test).

6.2 Scenario evaluation of the delay and risk engine

The engine was run on four project scenarios, matching the demonstration data, plus two edge cases. Each scenario has a planned schedule and progress records at four-week intervals. Results are in Table 3 and Figure 8. Repeating the computation 1,000 times on the same input gave identical outputs, confirming that the engine is deterministic.

Table 3: Engine output on evaluation scenarios

Scenario	Recs	Dev.	Actual / required %/wk	Velocity ratio	Predicted delay	Risk (band)	Baseline
Commercial office (slipping)	5	−10	2.00 / 3.00	0.67	70 days	72 (HIGH)	MEDIUM
Residential tower (on plan)	4	−1	1.67 / 1.68	1.00	2 days	23 (LOW)	LOW
Retail complex (ahead)	3	+6	4.88 / 3.67	1.33	0 (21 days early)	10 (LOW)	LOW
Metro annex (stalled)	4	−21	1.17 / 3.85	0.30	321 days	91 (CRITICAL)	CRITICAL
Single record (edge)	1	−2	—	—	—	Insufficient data	LOW
Two records, exact pace (edge)	2	0	5.00 / 5.00	1.00	0 (on time)	15 (LOW)	LOW

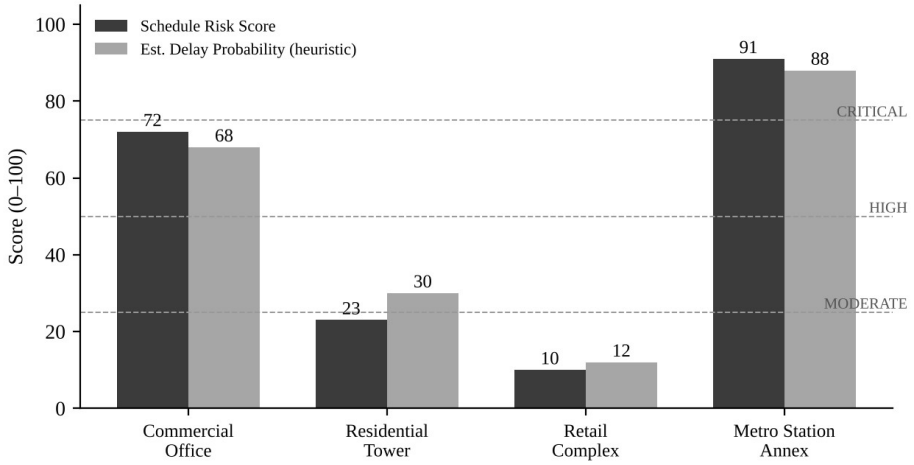


Figure 8: Schedule-risk score and delay-probability heuristic per scenario

Discussion. The engine separates the scenarios clearly and in the expected order. The comparison with the deviation-only baseline shows why several factors are combined. For the commercial office, the current deviation of −10 points alone gives only MEDIUM risk. The multi-factor engine rates it HIGH because the observed pace (2.0 %/week) is only two-thirds of the pace needed to finish on time and the deviation has been widening, which together project a 70-day overrun. For the stalled site, both methods agree on CRITICAL, and the engine also gives an actionable projected delay. With only one record, the engine refuses to predict, whereas a deviation-only rule still outputs a label.

Limitations. These scenarios check consistency and explainability; they do not measure predictive accuracy. Calibrating the weights against historical project outcomes requires completed-project data, which is listed as future work. YOLO11n has not been benchmarked on a labelled construction dataset.

7. Deployment

The system is deployed entirely on free tiers. The Next.js application runs on Vercel, with the build script generating the Prisma client and applying migrations on every deployment. PostgreSQL and image storage are on Supabase, using a private bucket with short-lived signed URLs. The YOLO11 service runs as a Docker container on a Hugging Face Space (CPU), protected by a shared API key. Analysis results are persisted, so earlier analyses remain viewable even while the AI service is idle. Step-by-step instructions are in docs/DEPLOYMENT.md in the repository.

Repository: https://github.com/yash06rajput/ConstructTrack

Live demo: _____

8. Research Paper Status

Working title: "An Explainable, Data-Sufficiency-Aware Framework for Construction Schedule-Risk Scoring with Computer Vision as Supporting Evidence".

Status: first full draft complete in IEEE two-column format (abstract, introduction, related work, methodology, system design, evaluation, discussion, limitations, conclusion). A hard copy accompanies this report.

Target venues (to be confirmed with the faculty guide):

- 3rd IEEE International Conference on Computer, Communication and Control (IEEE-IC4 2027), Medicaps University, Indore, 18–20 March 2027 — proceedings submitted to IEEE Xplore. Listed full-paper deadline: 15 October 2026; check for an extension.
- Journal alternative: ITcon — Journal of Information Technology in Construction (open-access, construction-informatics scope).

9. Remaining Work and Timeline

Task	Target
Incorporate third-defense feedback; finalise paper and submit to the chosen venue	Within 2 weeks
Collect a small set of real site photographs; report qualitative detection examples in the paper	Within 3 weeks
User walkthrough with 2–3 civil-engineering students or site staff; record feedback	Within 4 weeks
Final project report, presentation and external viva preparation	Before end-semester viva

10. Future Scope

- Authentication and multi-user, multi-organisation access control.
- A construction-specific detection model (for example, fine-tuned on labelled site imagery) behind the existing model-adapter interface.
- Calibrating risk weights, or fitting a statistical model, once outcome data from completed projects is available.
- Automated alerts when a project enters the HIGH or CRITICAL band, and charts embedded in the PDF report.
- Productisation: the team has begun BuildBridge, a multi-tenant evolution with organisations, roles, invitations and a tower → floor → stage progress model designed for real developers and site engineers.

References

[1] G. Jocher, J. Qiu and A. Chaurasia, "Ultralytics YOLO," 2023–2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, 2014, pp. 740–755.

[3] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed. Newtown Square, PA, USA: PMI, 2021.

[4] S. A. Assaf and S. Al-Hejji, "Causes of delay in large construction projects," Int. J. Project Management, vol. 24, no. 4, pp. 349–357, 2006.

[5] Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs

[6] Prisma, "Prisma ORM Documentation." [Online]. Available: https://www.prisma.io/docs